

## # Welcome

Thank you for participating in our technical assessment process. We're excited to see what you build!

This project is designed to evaluate your ability to design and implement a multi-channel notification delivery system. Take your time, think through the reliability requirements, and show us how you approach complex problems.

---

## ## About This Assessment

You'll be building a notification service that delivers messages across multiple channels (email, SMS, push, webhook) with proper queuing, retry logic, and delivery tracking. This is a common infrastructure component in modern applications.

We're interested in seeing how you handle delivery reliability, channel abstraction, and user preferences. There's no single "right" answer - we want to see your thought process and trade-offs.

---

## ## Using AI Tools

You are **explicitly permitted and encouraged** to use AI coding assistants during this assessment. This includes ChatGPT, GitHub Copilot, Claude, Cursor, or any other AI tools you prefer.

**\*\*Why we allow this:\*\***

- AI tools are part of modern development - we use them too
- Notification systems have subtle reliability requirements that AI often misses
- We're evaluating your ability to critically assess AI suggestions

**\*\*What we're actually evaluating:\*\***

- Do you understand delivery guarantees and failure handling?
- Can you design a clean abstraction for multiple channels?
- Do you recognize when AI gives you subtly incorrect advice?
- Can you explain the trade-offs in your design?

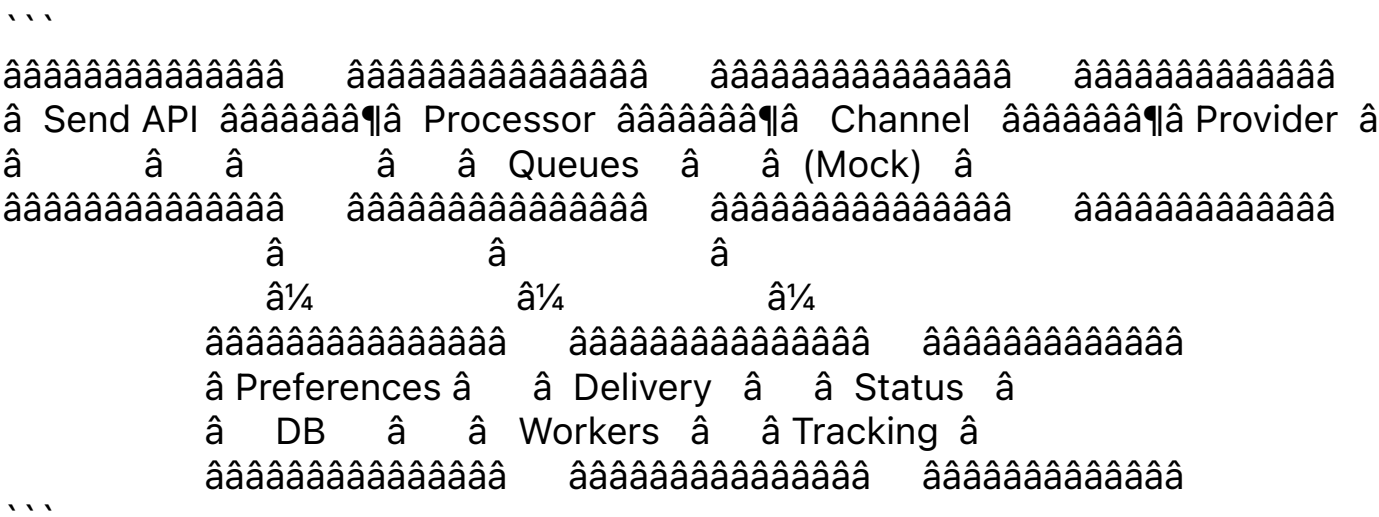
**\*\*Important:\*\*** You will be asked to explain your architecture and handle "what if" scenarios in a follow-up interview. Make sure you deeply understand your solution.

---

## System Overview

Build a notification service that:

- 1. **Receives notification requests** via API
- 2. **Processes user preferences** to determine channels
- 3. **Queues delivery tasks** per channel
- 4. **Delivers via channel providers** (mocked)
- 5. **Tracks delivery status** and handles failures



## Functional Requirements

### Send Notification

- Accept notification requests with:
- Recipient (user ID or contact info)
  - Notification type (e.g., "order\_confirmation", "password\_reset")
  - Content (or template reference with variables)
  - Priority (high, normal, low)
  - Channels override (optional - use specific channels instead of preferences)
  - Scheduled time (optional - send later)

### User Preferences

- Users can configure:
- Enabled channels (email, SMS, push, webhook)
  - Per-type preferences (e.g., marketing via email only, alerts via all channels)
  - Quiet hours (don't send during these times, except high priority)
  - Frequency caps (max N notifications per hour/day)

### ### Channel Delivery

Implement these channel providers (all mocked):

#### **\*\*Email\*\***

- Send to email address
- Support HTML and plain text
- Track: sent, delivered, opened, bounced

#### **\*\*SMS\*\***

- Send to phone number
- Character limit handling
- Track: sent, delivered, failed

#### **\*\*Push Notification\*\***

- Send to device token
- Support title + body + data payload
- Track: sent, delivered, clicked

#### **\*\*Webhook\*\***

- POST to configured URL
- Include notification payload
- Track: sent, acknowledged, failed

### ### Templates

- Create notification template (type, channel, subject, body with variables)
- Templates support variable substitution: `Hello {{user.name}}`
- Different template per channel for same notification type

### ### Delivery Tracking

- Get notification status (pending, processing, delivered, failed)
- Get per-channel delivery status
- Get delivery history for a user
- Get aggregate stats (sent, delivered, failed by channel)

### ### Management

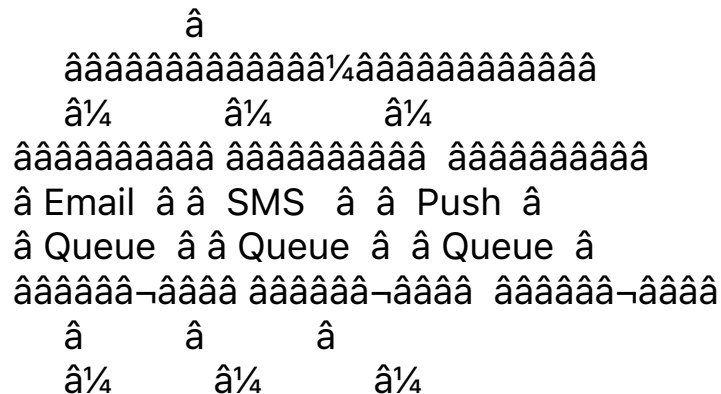
- Resend failed notification
- Cancel scheduled notification
- Pause/resume notifications for a user

---

## ## Notification Lifecycle

...

RECEIVED → PROCESSING → QUEUED (per channel)



SENDING → DELIVERED / FAILED

(retry if failed)

PERMANENTLY\_FAILED  
(after max retries)

...

---

## ## Data Model

Design your data model to support:

### ### User Preferences

- User identifier
- Global enabled channels
- Per-type channel preferences
- Quiet hours configuration
- Frequency cap settings
- Webhook URL (for webhook channel)

### ### Template

- Notification type
- Channel
- Subject (for email)
- Body (with variable placeholders)
- Active flag

### ### Notification

- Unique identifier

- Recipient (user ID or contact info)
- Notification type
- Content/template reference
- Variables (for template substitution)
- Priority
- Status (received, processing, completed, failed)
- Scheduled at (optional)
- Created at

### ### Delivery

- Notification relationship
- Channel
- Recipient address (email, phone, device token, URL)
- Status (queued, sending, delivered, failed)
- Attempts count
- Last attempt at
- Delivered at
- Error message (if failed)
- Provider response (for debugging)

---

## ## Critical Implementation Details

### ### 1. Channel Abstraction

Design a clean interface so adding new channels is straightforward:

- Common interface for all providers
- Channel-specific configuration
- Consistent status tracking

**\*\*Document your approach in DECISIONS.md\*\***

### ### 2. Delivery Reliability

When a delivery fails:

- Which failures should retry? (network timeout vs invalid address)
- Retry timing (immediate, backoff, scheduled)
- Max retry attempts per channel
- When to mark permanently failed

**\*\*Document your approach in DECISIONS.md\*\***

### ### 3. Preference Resolution

When sending a notification:

1. Check user's global preferences
2. Check type-specific preferences
3. Apply quiet hours rules
4. Check frequency caps
5. Apply channel override if specified

What's the priority order if these conflict?

**\*\*Document your approach in DECISIONS.md\*\***

### ### 4. Multi-Channel Coordination

If sending to multiple channels:

- Send in parallel or sequence?
- If one channel fails, does it affect others?
- How do you report overall status?

**\*\*Document your approach in DECISIONS.md\*\***

### ### 5. Frequency Caps

Prevent notification fatigue:

- Track sent count per user per time window
- Respect caps even for high priority?
- What happens to capped notifications? (drop, queue, alert)

---

## ## Technical Requirements

### ### Must Have

- Python 3.11+
- Web framework of your choice
- Database for persistence
- Queue mechanism for async delivery
- At least 3 channel providers implemented (mocked)
- Retry logic with backoff
- User preference support
- Containerized setup (docker-compose)
- At least 6 meaningful tests:
  - Send notification end-to-end
  - Preference filtering
  - Retry on failure

- Template rendering
- Multi-channel delivery
- Status tracking

### ### Should Have

- Template variable substitution
- Scheduled notifications
- Quiet hours support
- Structured logging with notification context

### ### Nice to Have

- Frequency caps
- Delivery analytics/stats endpoint
- Batch send (multiple recipients)
- Priority queues (high priority processed first)

---

## ## Submission Requirements

Submit a Git repository containing:

```

```
your-project/
  README.md
  AI_USAGE.md
  DECISIONS.md
  docker-compose.yml
  Dockerfile
  (your application code)
  (your tests)
```

```

### ### README.md

Include:

1. How to run the project
2. How to run tests
3. Example API calls for sending a notification
4. Which channels you implemented

### ### DECISIONS.md (Required)

```markdown

## # Design Decisions

### ## 1. Channel Abstraction Design

**\*\*Approach chosen:\*\*** [Your interface/pattern]

**\*\*Why:\*\*** [Your reasoning]

**\*\*Adding a new channel requires:\*\*** [What changes]

---

### ## 2. Retry Strategy

**\*\*Approach chosen:\*\*** [Your approach]

**\*\*Retryable vs non-retryable errors:\*\*** [How you distinguish]

**\*\*Timing:\*\*** [Backoff strategy]

---

### ## 3. Preference Resolution Order

**\*\*Approach chosen:\*\*** [Your priority order]

**\*\*Conflicts resolved by:\*\*** [Your rules]

**\*\*Example:\*\*** [Walk through a scenario]

---

### ## 4. Multi-Channel Coordination

**\*\*Approach chosen:\*\*** [Parallel/sequential/hybrid]

**\*\*Why:\*\*** [Your reasoning]

**\*\*Failure isolation:\*\*** [How one channel's failure affects others]

---

### ## 5. One Thing I Would Do Differently With More Time

[Be honest - what did you skip or simplify?]

```

### AI\_USAGE.md

```markdown

# AI Tool Usage

## Tools I Used

[List the AI tools you used]

## What Helped Most

[Describe 1-2 specific cases where AI helped significantly]

## What I Had to Fix

[Describe 1-2 cases where AI gave incorrect advice]

## What AI Struggled With

[Any parts where AI wasn't helpful]

```

---

## Tips for Success

1. **\*\*Get one channel working end-to-end first\*\*** - Prove the flow
2. **\*\*Abstract early\*\*** - Design the channel interface before adding more channels
3. **\*\*Mock providers simply\*\*** - Just log + random success/failure
4. **\*\*Test failure paths\*\*** - Retry logic is where bugs hide
5. **\*\*Be explicit about simplifications\*\*** - We value honesty

---

We look forward to seeing your solution and discussing it with you!